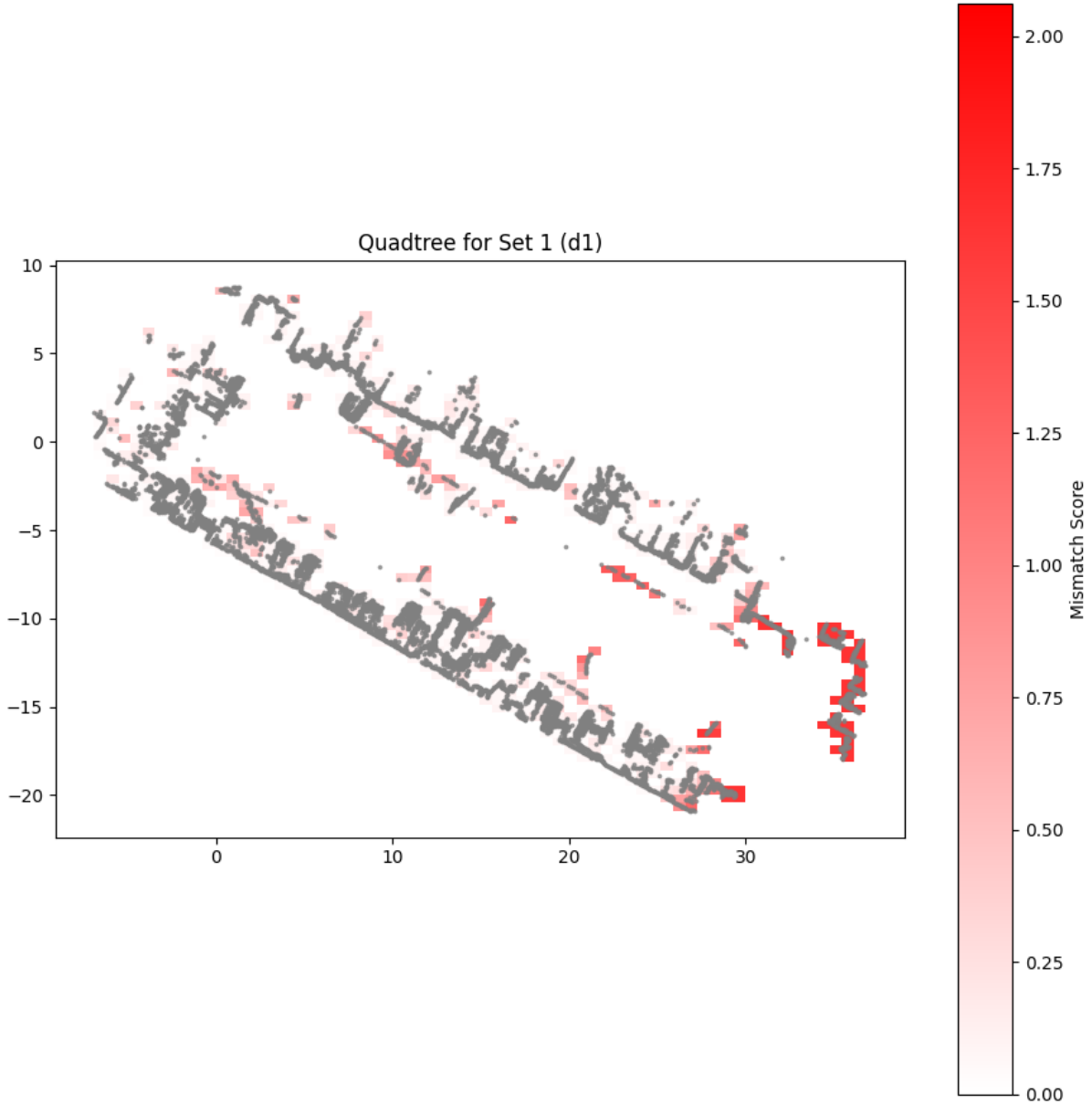
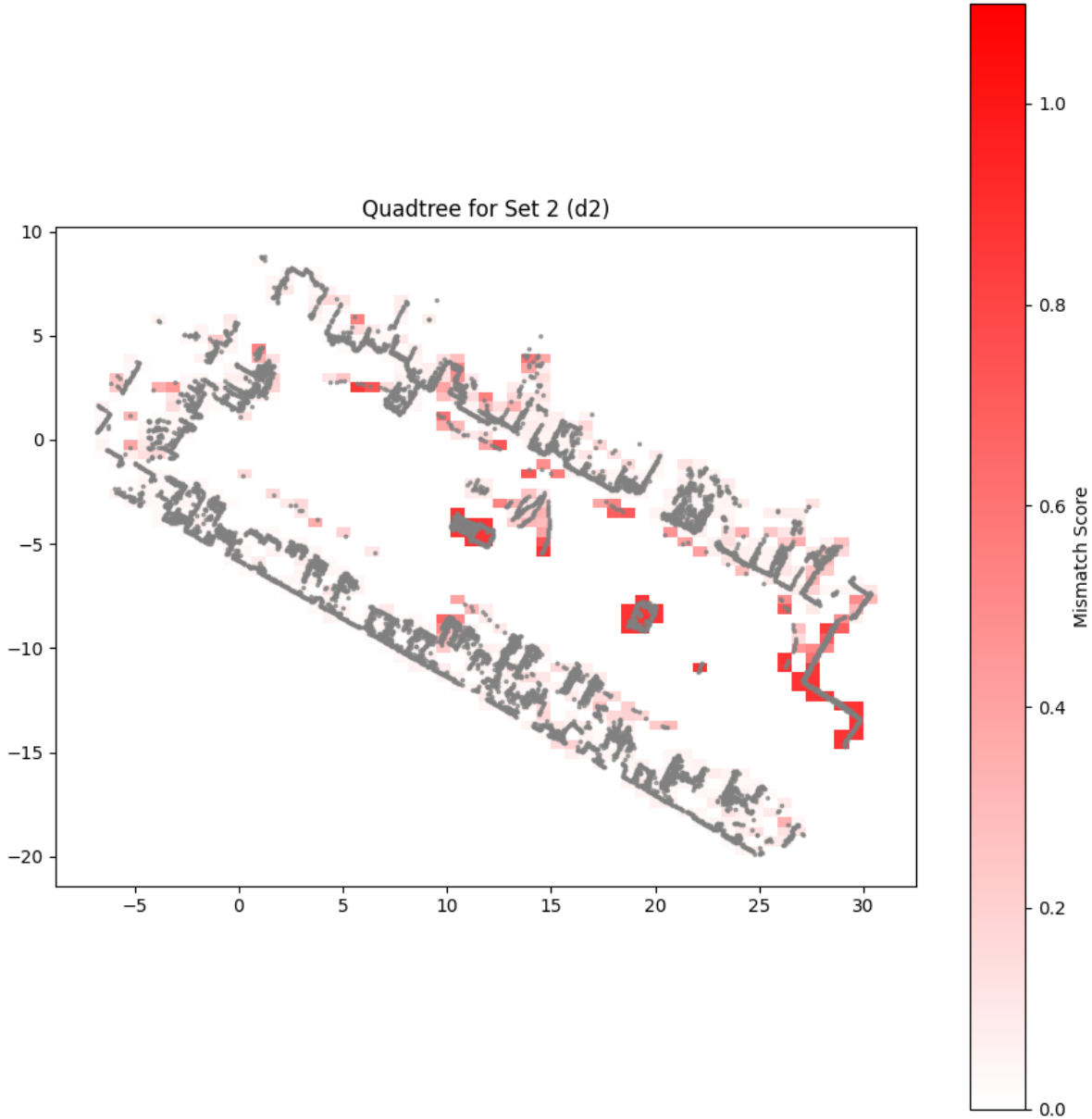
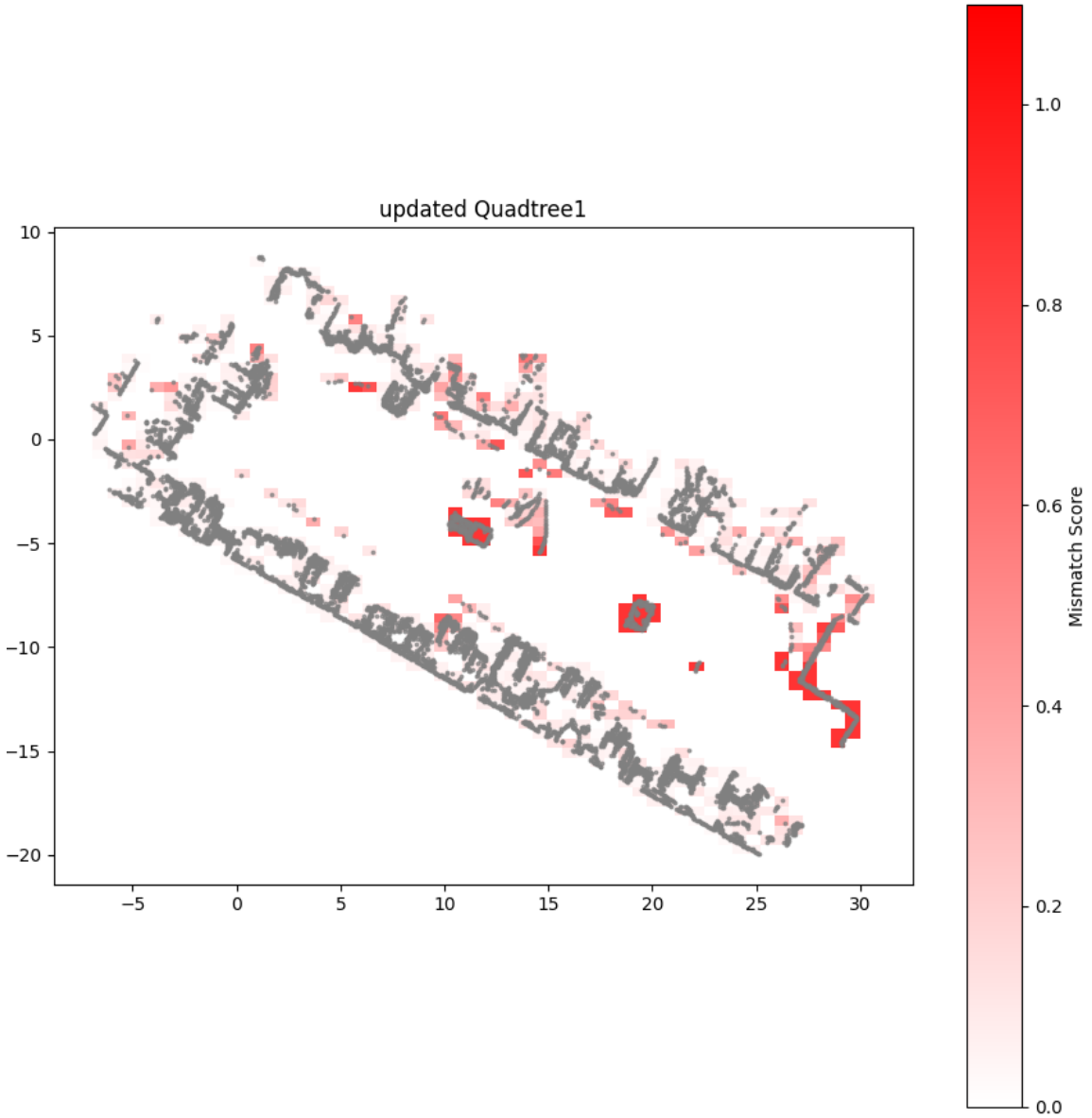
Change detection

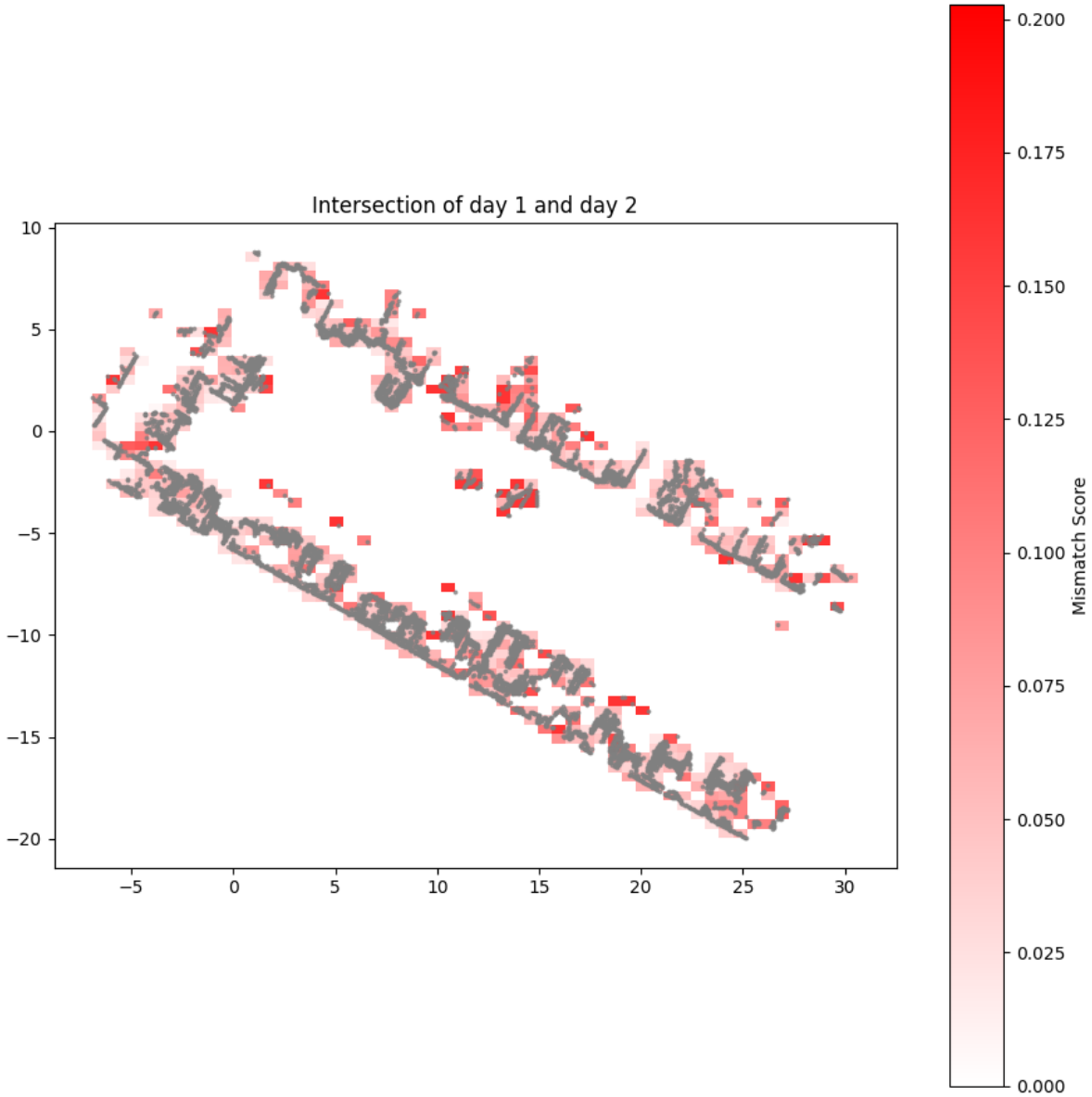
The 1st image shows the data from day 1.
The 2nd image shows the data from day 2.



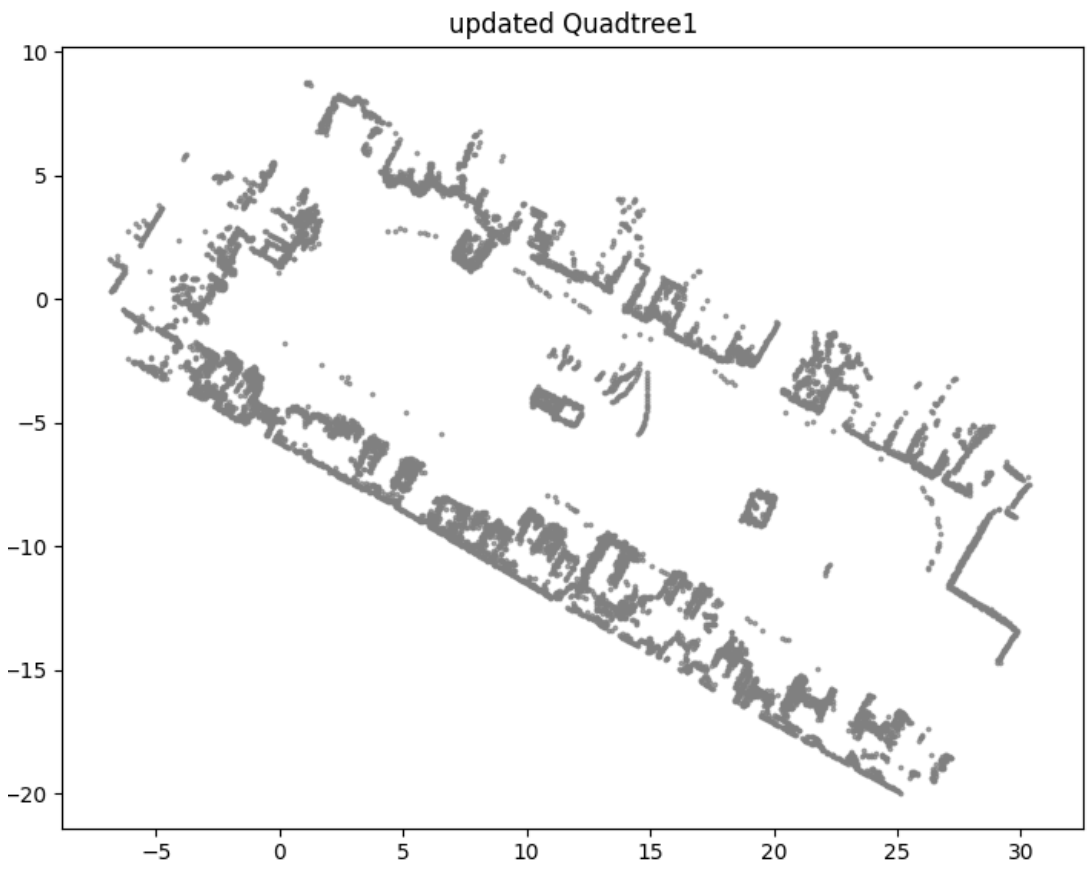


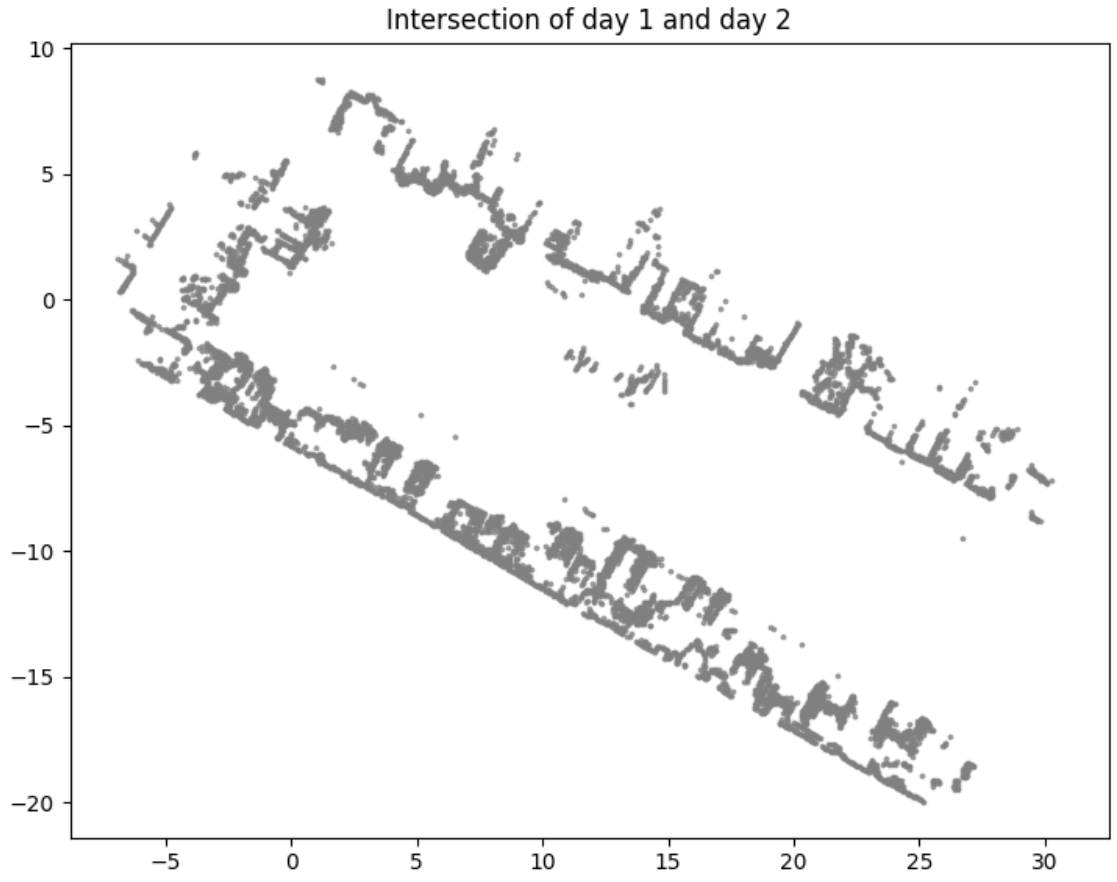
The 1st image shows the quadtree from day 1 updated such that each quad is replaced with the corresponding quad from day 2, if the two differ significantly.
The 2nd image shows the intersection between day 1 and day 2 (all the things that didn't change)





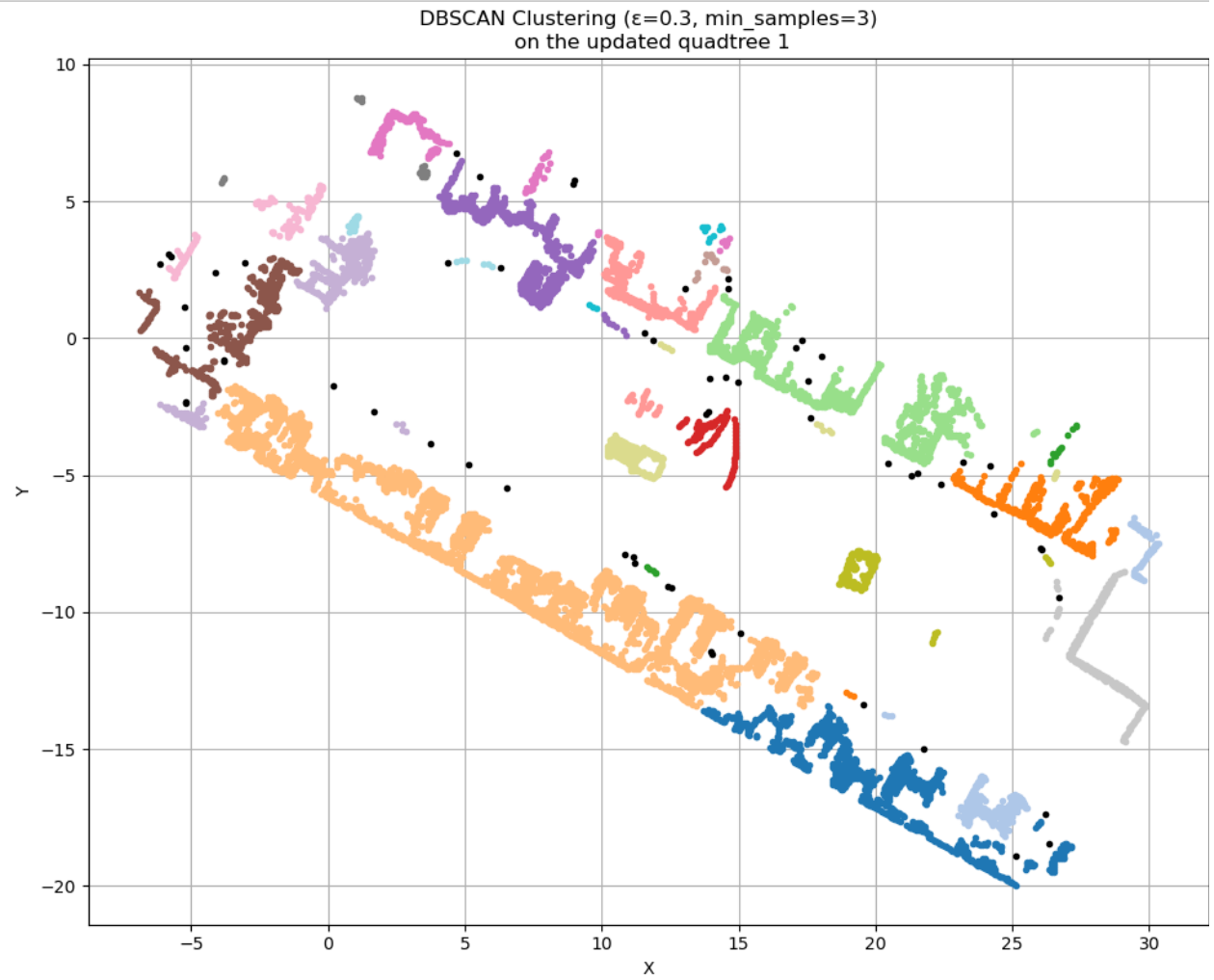
Cleaner visualization





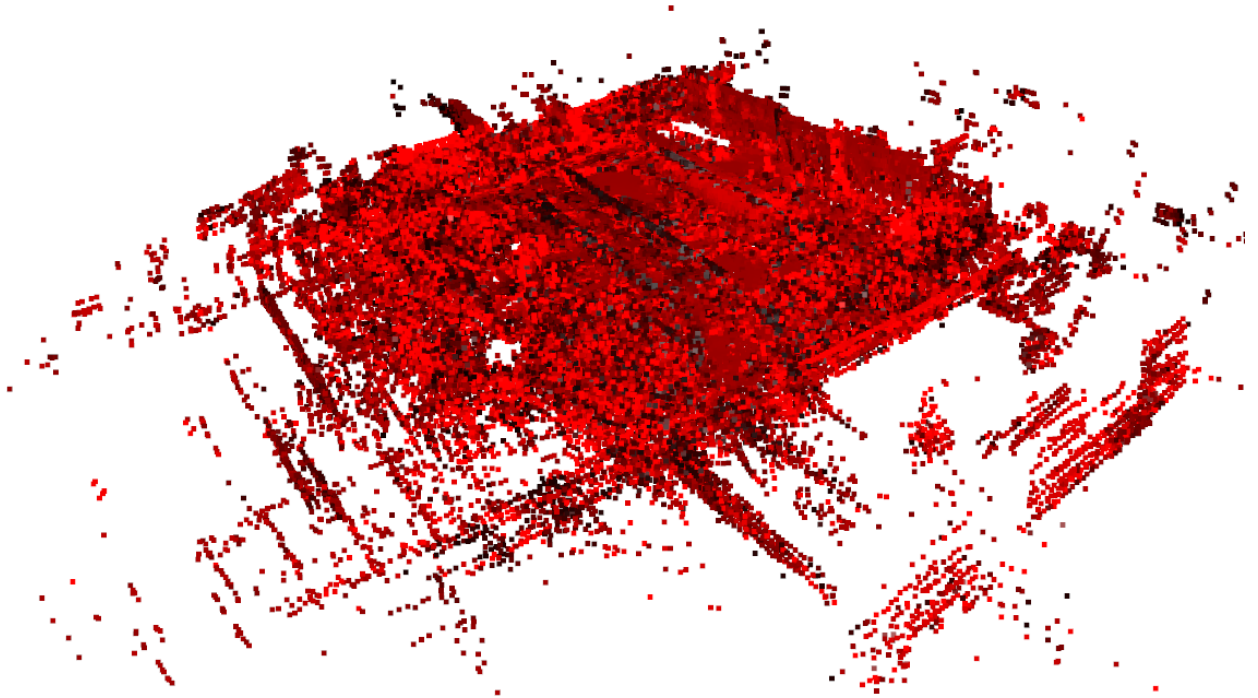
DBscan of the updated quadtree: (1) group points into possible objects , (2) identify Noise and Outliers .

-- adjusting the parameters gives us different levels of detail/resolution in object recognition.



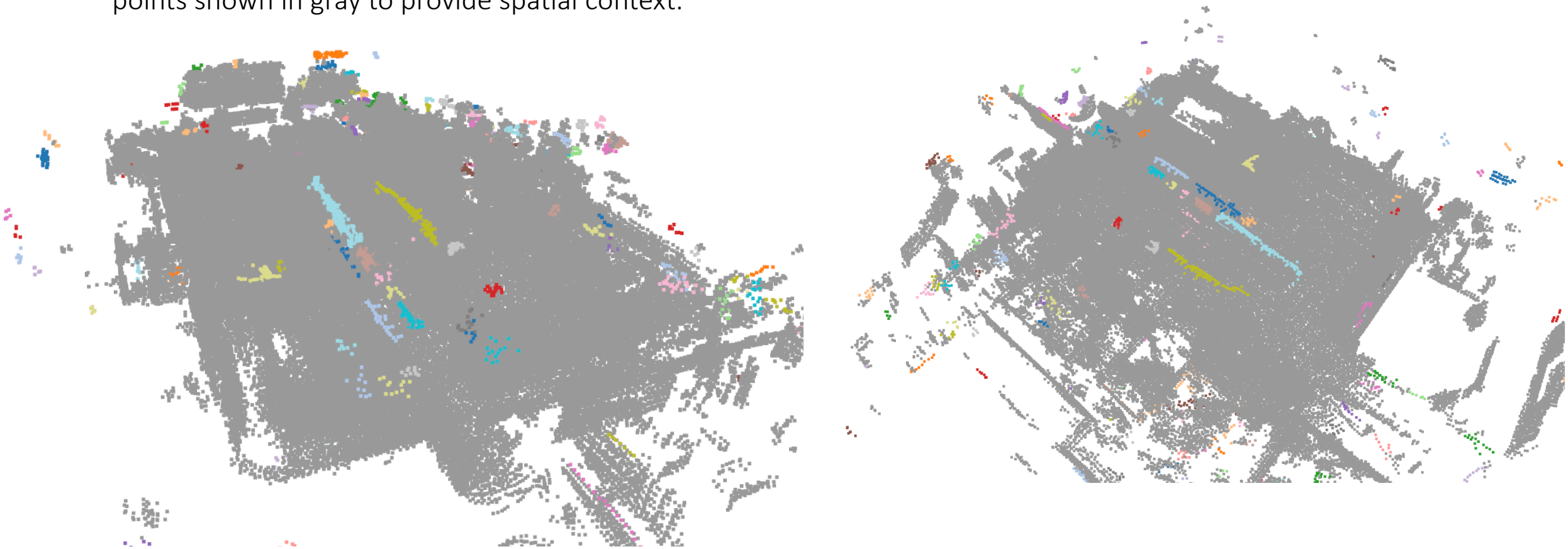
3d point clouds

the 2 point clouds together



Changes in 3d points

1) Detects changes between two 3D point clouds by computing nearest-neighbor distances and filtering points that moved beyond a defined threshold.
2) Segments the changed points using DBSCAN clustering, identifying spatially grouped areas of significant change.
3) Visualizes both changed and unchanged points, with changed points color-coded by cluster and unchanged points shown in gray to provide spatial context.



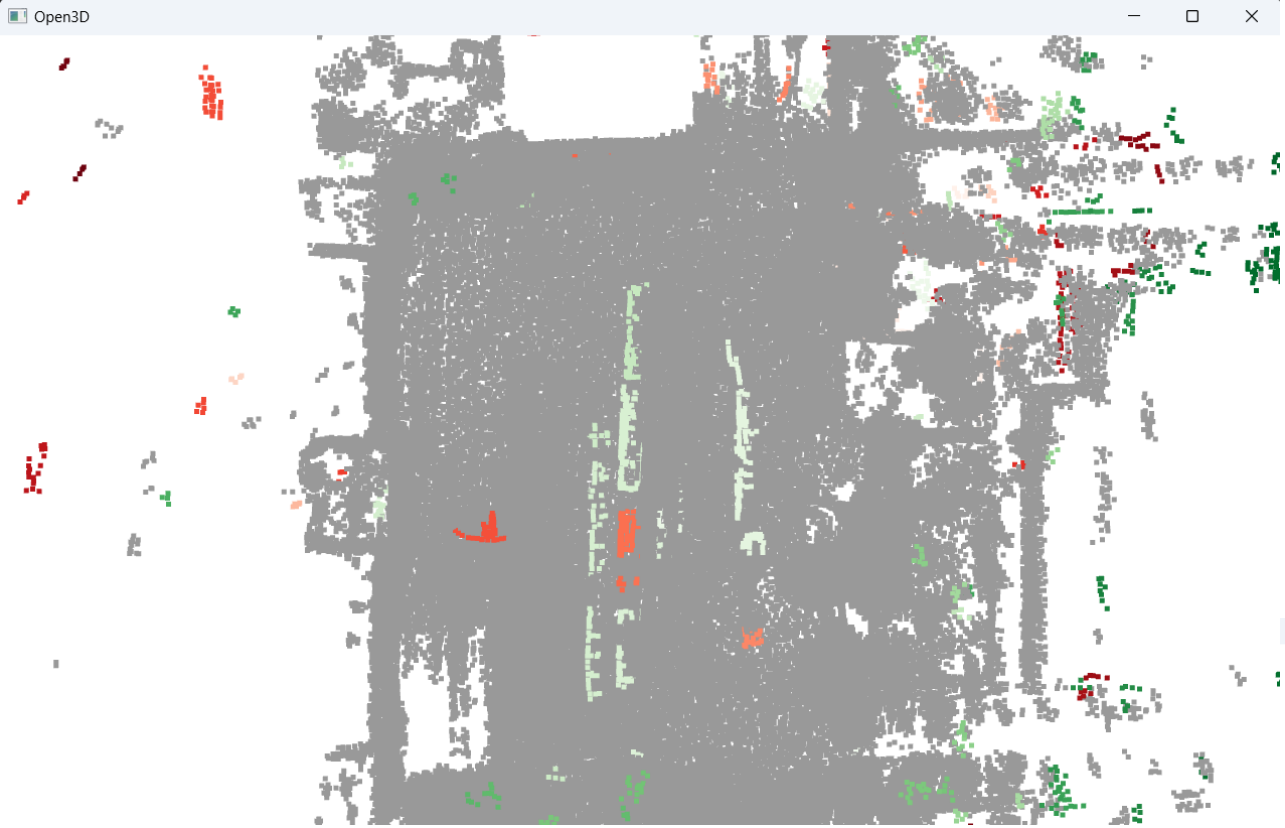
Changes in 3d points (better visualization)

- keep DBSCAN clustering, but instead of coloring all clusters with a shared palette:

  Clusters formed from changed_points1 (points that only existed in time1) → colored with a red-based colormap.

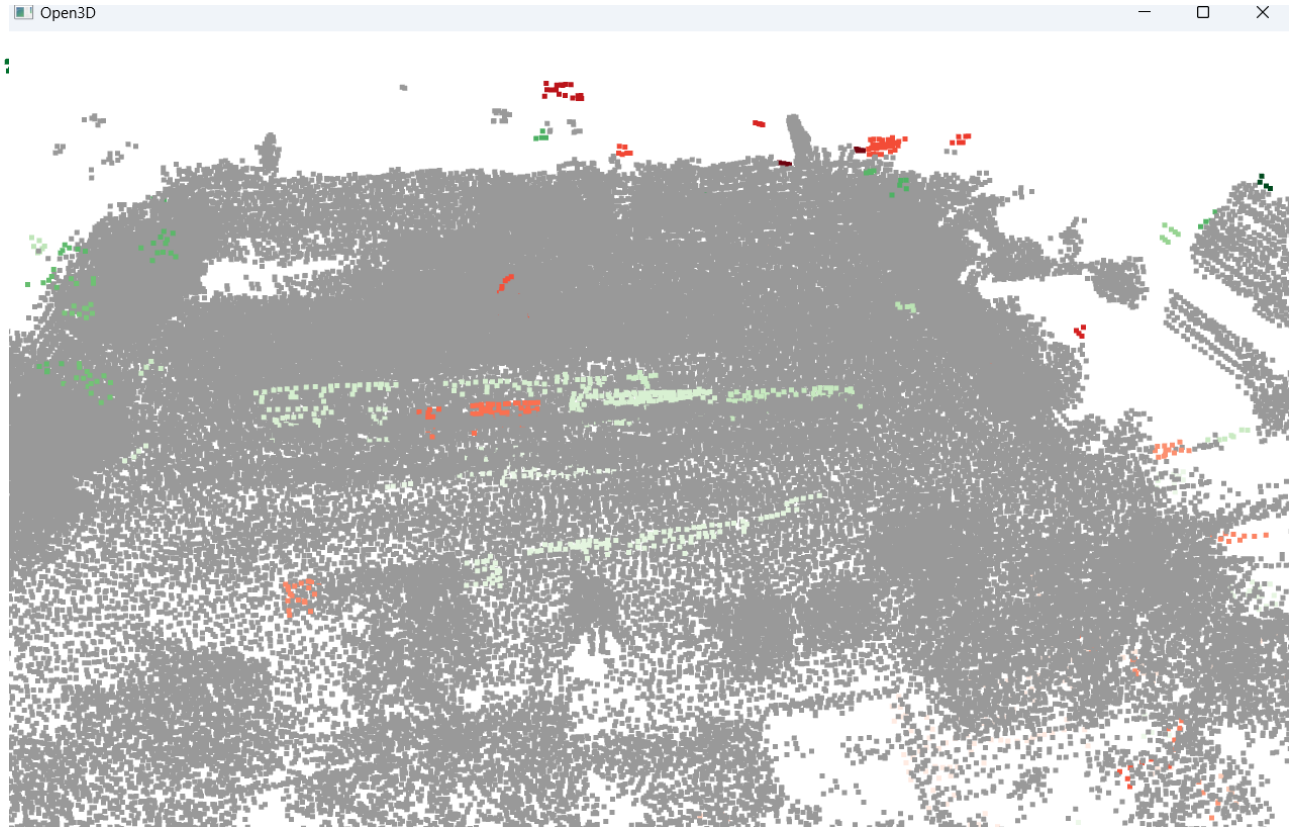
  Clusters formed from changed_points2 (points that only existed in time2) → colored with a green-based colormap.

- That way, you can still distinguish clusters,
- red hues = disappeared objects,
- green hues = new objects.



Detected 225 total change clusters.

- 81 clusters from time1 (disappeared → red hues)
- 144 clusters from time2 (appeared → green hues)
- 2153 noise points



Change Detection in 3d pointclouds from dataset: “warehouse_3d_capture/same_area_scans_different_intervals”

- Three distinct approaches were implemented and evaluated:
  1. Raw Registration (GICP): Aligning scans using pure geometry (Generalized ICP) without filtering. The result is cluttered with static noise (walls, floors), making it difficult to isolate inventory changes.
  2. Geometric Filter (RANSAC): Mathematically detects and removes flat planes (trying to remove the walls/floor). However, this method is "context-blind"—it often fails on uneven ground or accidentally deletes flat objects (like pallets) that it mistakes for the floor.
  3. Semantic AI (Cylinder3D) - *Proposed Method*: Utilizes Deep Learning to semantically *understand* the scene. It intelligently filters infrastructure (walls, road) while preserving inventory, delivering the cleanest and most accurate change detection.

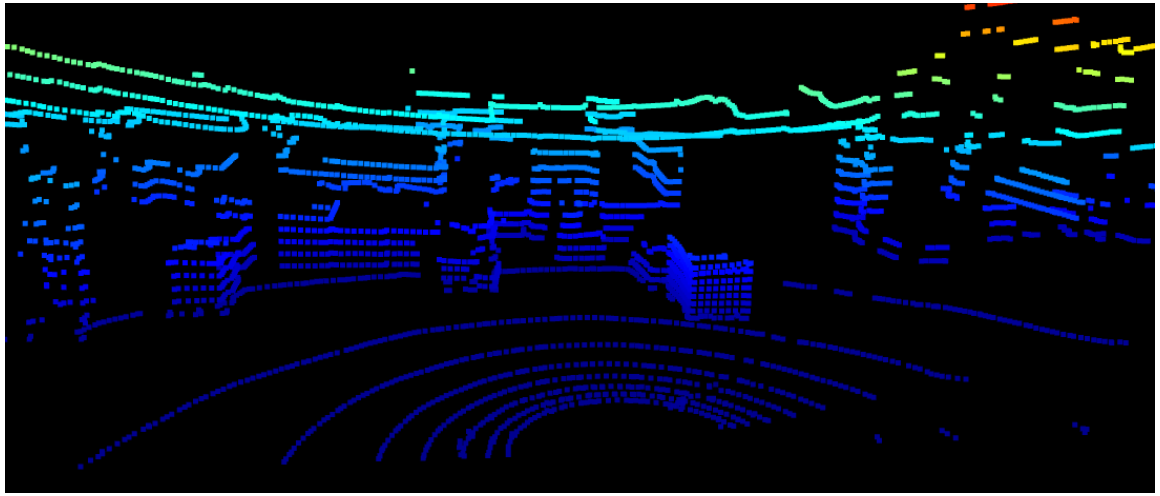


Figure 1. One of the 3 scans.

Registration Pipeline Overview

Global Alignment (RANSAC + FPFH)

Point clouds are first voxel-downsampled and described using FPFH features.
A RANSAC-based matcher estimates a coarse rigid transformation, robust to large initial misalignment.

Local Refinement (Multi-Scale GICP)

The coarse alignment is refined using Generalized ICP across multiple voxel resolutions (1.0 → 0.1 m).
This coarse-to-fine strategy improves convergence stability and registration accuracy.

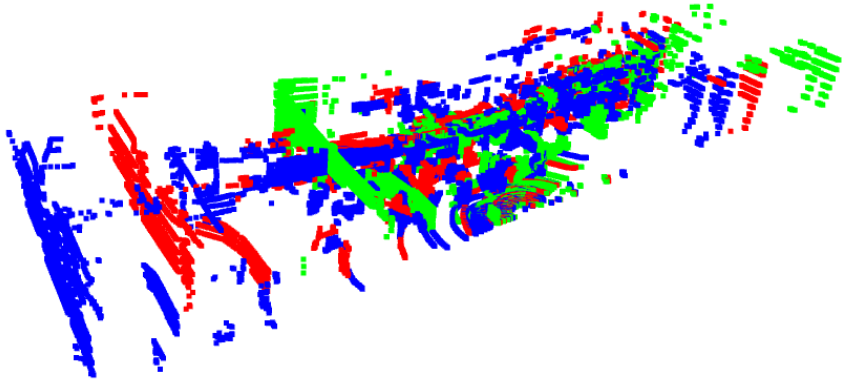


Figure 2. 3 scans Before registration.

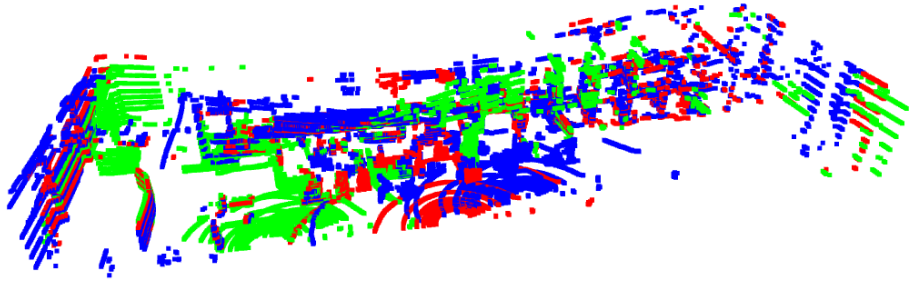
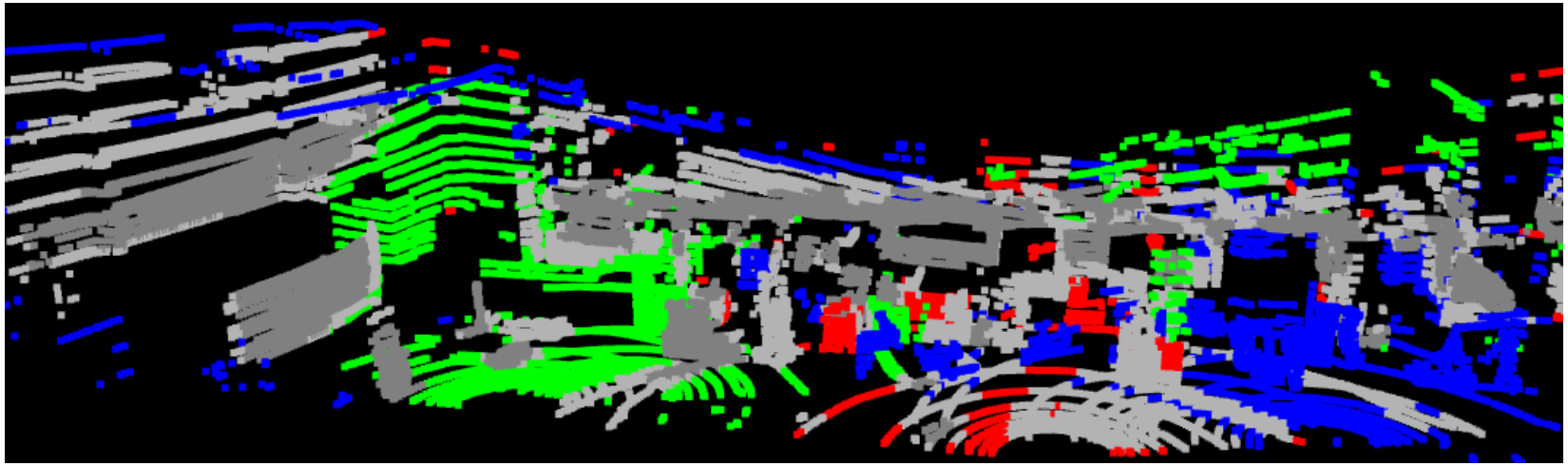


Figure 3. 3 scans After registration.

1st method: Geometric approach

- **Logic:** The script uses a **KD-Tree Radius Search.** For every point in *SCAN X*, it looks for neighbors in *all other scans* within a radius of “threshold”.
- **If neighbors exist in all scans:** The point is marked **Grey** (Static).
- **If neighbors are missing:** The point is marked **Red/Green/Blue** (according to in which scan it exists) (Change)



- Why simple geometric comparison fails in dynamic environments:
- **Occlusion Artifacts (Ghosting):** Removing an object reveals previously hidden surfaces (shadows). These are often misinterpreted as "new" geometry, creating False Positives.
- **Viewpoint Sampling Noise:** Shifts in LiDAR position cause non-identical sampling of static surfaces (walls/floors), generating false changes due to misalignment or sparsity.
- **Lack of Semantic Context:** Without segmentation, the system cannot distinguish inventory from infrastructure, causing it to mix background noise with actual object changes.

2nd method: Geometric Segmentation (RANSAC)

- Concept:

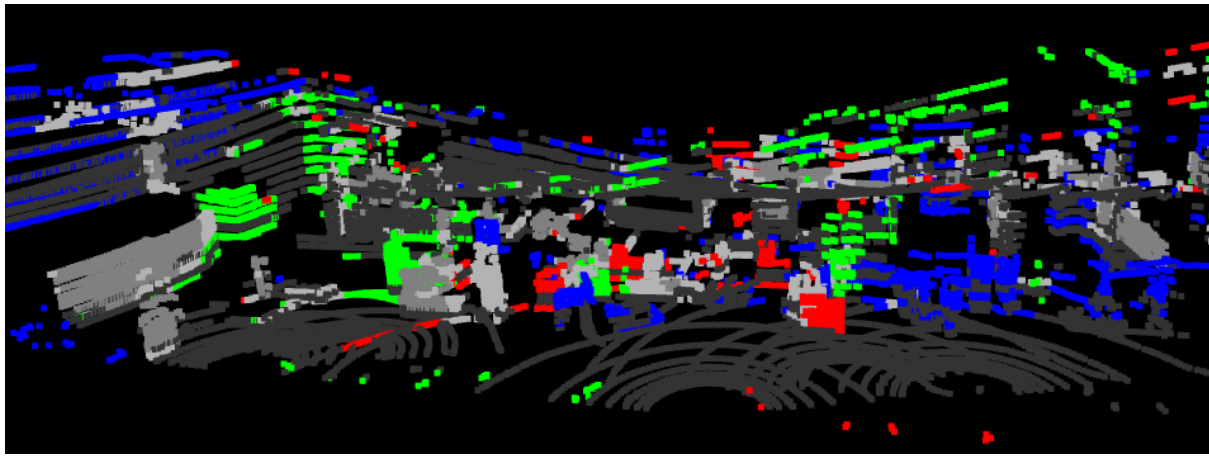
Attempts to solve the "noise" problem of the first method by mathematically detecting and removing the background before performing change detection. It assumes that the floor and walls are the largest flat surfaces in the scene.

- How it Works:

Plane Fitting (RANSAC): The algorithm randomly samples points to find the largest mathematical plane (e.g., the floor).

iterative Removal: It repeats this process to find and remove secondary planes (e.g., walls), isolating the "Background" (Static).

Filtered Comparison: Change detection is only performed on the remaining non-planar points (Objects), theoretically reducing false positives from walls.



Better result. It segments out parts of wall and the floor But the result still has some of the problems of the 1st approach.

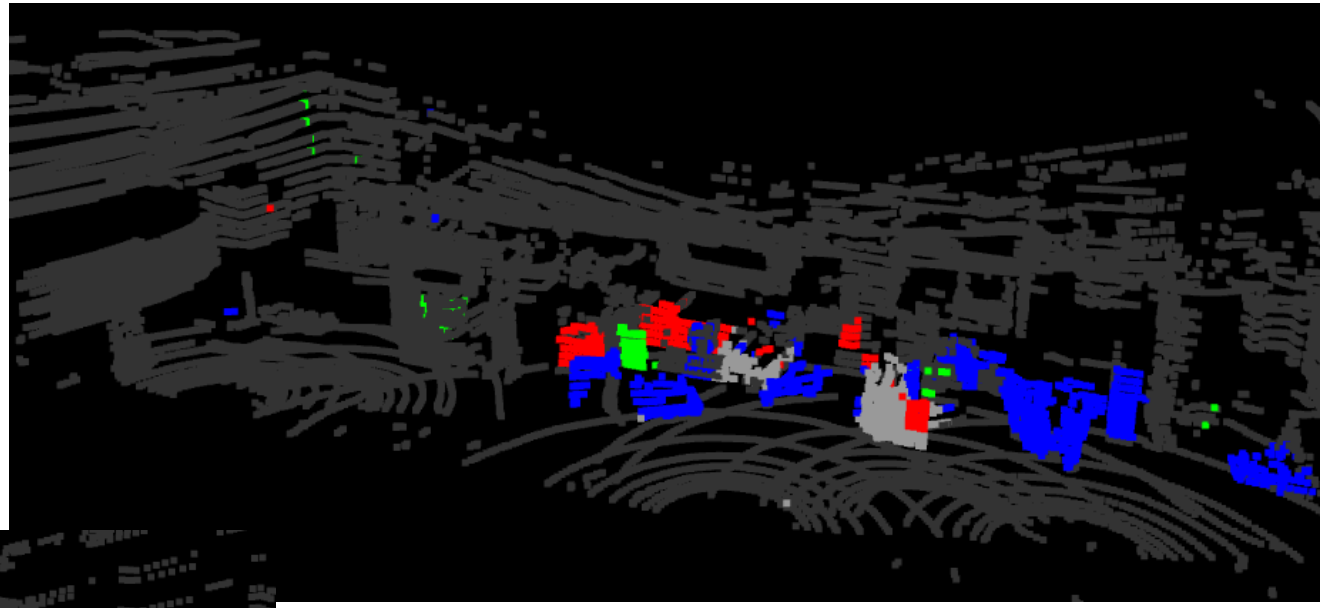
3rd method: Semantic AI Segmentation (Proposed)

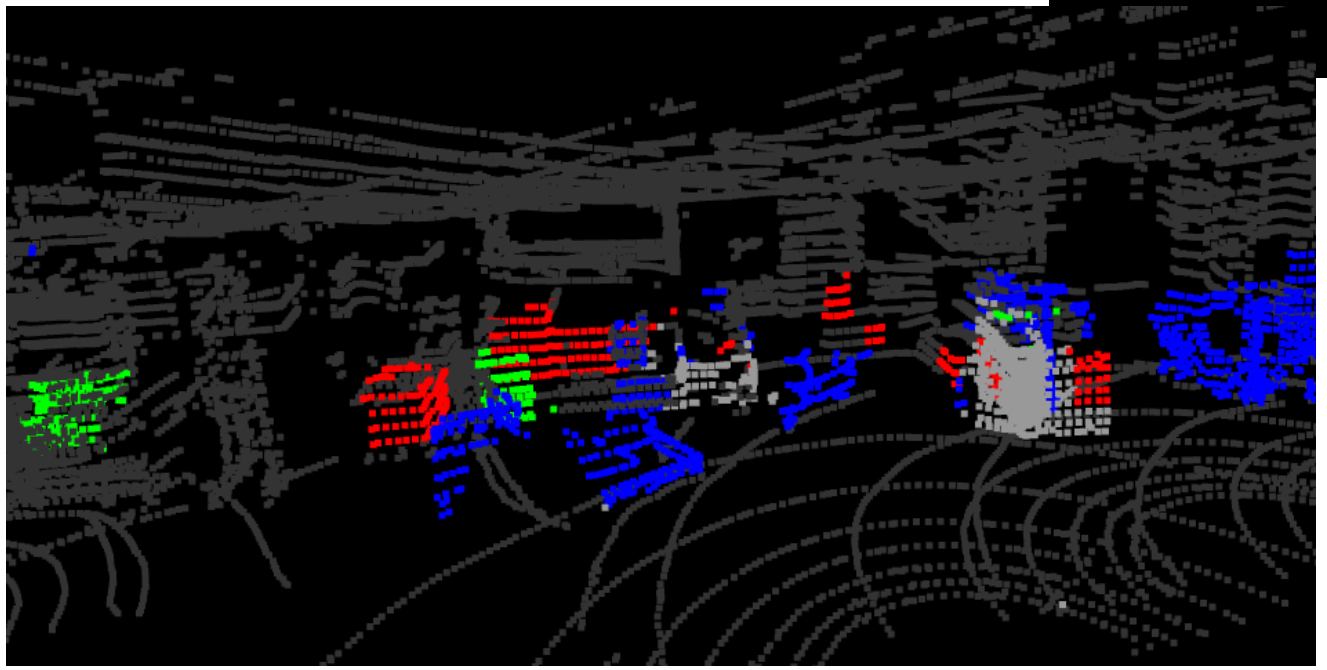
- Concept:
- Unlike the previous methods that rely on "blind" geometry, this approach uses **Deep Learning** to semantically *understand* the scene. By classifying every point (e.g., "Wall", "Floor", "Box"), we can intelligently **filter out the permanent infrastructure while strictly monitoring the inventory**.
- How it Works:
- **Deep Learning Inference:** The raw point cloud is passed through the **Cylinder3D** neural network, which assigns a semantic label (0-19) to every single point.
- **The "Smart" Filter:** We apply a configuration to isolate inventory from infrastructure:
  - **REMOVE (Infrastructure):** We filter out **Floor** classes (8, 9, 10, 11, 16) and **Buildings/Walls** (Class 12). This removes the permanent warehouse shell.
  - **KEEP (Inventory):** We retain dynamic classes like **Fences/Cages** (Class 13), **Vegetation** (14), and **Misc** objects.
- **Context-Aware Change Detection:** The change detection algorithm runs *only* on the remaining points, effectively comparing "Box vs. Box" instead of "Box vs. Wall."

3rd method: Semantic AI Segmentation (Proposed)

Why it is the Best:

Solves Ghosting: It recognizes that the wall behind a moved box is just a "Wall" (infrastructure), so it ignores it instead of flagging it as a change.
Ignores Noise: Sampling noise on the floor is irrelevant because the entire floor class is semantically removed.





It really detects changes in the inventory of the warehouse!

Results video